

Bash, automatización y procesamiento de texto

TC-CH02 – Técnicas Computacionales en Biología

Edición 2 · 2026-09-03

Pregunta del tema. Ha ejecutado a mano la misma orden sobre veinte ficheros y mañana llegan veinte más. ¿Cómo se convierte esa secuencia de órdenes en un procedimiento que otra persona pueda ejecutar, leer y corregir?

Producto final. Un script propio, con sus comprobaciones y su registro, que recorre un corpus de ficheros biológicos y produce una tabla de resultados reproducible.

Mapa del tema

1. De la orden repetida al procedimiento automático	2
2. Fundamentos de Bash: órdenes, opciones y variables	2
3. Caracteres especiales y comillas	4
4. Flujos estándar: stdin, stdout, stderr	4
5. Redirección: escribir, añadir, leer y separar errores	5
6. Tuberías: componer programas sin que se conozcan entre sí	5
7. De la orden interactiva al script	6
8. Variables, comillas y argumentos posicionales	7
9. Aplicación: proteger una operación destructiva	8
10. Condicionales	9
11. Bucles	10
12. Funciones	11
13. Arrays y expansión de llaves	12
14. Operadores nativos de cadena	13
15. Hito de consolidación: del programa robusto al texto biológico	13
16. El formato FASTA como primer registro multilínea	14
17. Buscar con grep	15
18. Expresiones regulares	16
19. Utilidades clásicas de texto: cut, sort, uniq y tr	18
20. Transformar con sed	19
21. Procesar texto estructurado con awk	20
22. Lo que viene después: de procesar texto a razonar sobre su coste	22
23. Método y errores frecuentes	22
24. Síntesis de referencia	23
25. Glosario esencial	23
26. Fuentes y límites de uso	24

Cómo usar este tema

Este capítulo es largo porque cubre dos bloques que el temario separa y que en la práctica van juntos. El primero, **programar la Shell**, va de la sección [De la orden repetida al procedimiento automático](#) al hito de la sección [Hito de consolidación: del programa robusto al texto biológico](#): variables, comillas, flujos, condicionales, bucles y funciones. El segundo, **procesar texto**, va del hito al final: FASTA, grep, expresiones regulares, sed y awk. El hito es la frontera, y conviene detenerse ahí: hasta ese punto se aprende a escribir programas que no mienten; a partir de ahí, a leer datos que no están limpios.

Al terminar sabrá: escribir un script con sus opciones de seguridad y saber qué protegen y qué no; entrecomillar de forma que un nombre con espacio no rompa nada; encadenar programas por tuberías y detectar el fallo de una etapa intermedia; recorrer ficheros con bucles y arrays sin tropezar con los globs vacíos; y extraer campos de un FASTA, un GFF o un TSV con grep, sed y awk, sabiendo cuál de los tres conviene en cada caso.

Las marcas [ESENCIAL](#), [ESTUDIO](#) y [REFERENCIA](#) señalan qué bloques forman la ruta esencial, cuáles amplían y cuáles son de consulta. La ruta esencial son los dos bloques nominales sin sus apartados avanzados. La ruta de estudio añade *lookaround*, dialectos, arrays avanzados y el ejemplo trabajado de GFF. Escriba su predicción antes de ejecutar cada bloque de terminal.

1 De la orden repetida al procedimiento automático

ESENCIAL

En el primer capítulo aprendimos los cimientos del entorno Unix: cómo comunicarnos con el sistema operativo a través de una terminal de texto, cómo orientarnos en la jerarquía del sistema de archivos y cómo ejecutar programas con seguridad. Ese conocimiento nos permite resolver con soltura cualquier tarea puntual: inspeccionar un archivo, mover un conjunto de datos o lanzar una herramienta bioinformática.

Sin embargo, la investigación biológica moderna rara vez se enfrenta a un fichero aislado. En proyectos reales de genómica, transcriptómica o metagenómica, los datos de secuenciación masiva se distribuyen en decenas o cientos de muestras de secuenciación (*FASTQ*), con miles de secuencias (*FASTA*) y extensas tablas de anotación genómica (*GFF* o *VCF*). Escribir órdenes una por una a mano sobre veinte ficheros no solo es tedioso: es una fuente frecuente de errores y, sobre todo, un procedimiento que nadie puede reproducir. La orden tecleada a mano no deja registro. En cuanto cierra la sesión desaparece, y con ella la respuesta a la única pregunta que importa seis meses después: qué parámetros usó exactamente y sobre qué ficheros.

Una regla práctica de la computación científica: **si va a repetir un procedimiento más de dos veces, automatízelo**. No por elegancia, sino porque el script es a la vez el procedimiento y su documentación. Este capítulo sigue el orden del programa oficial de la asignatura (*Módulo 2: Shell Scripting y Procesamiento de Texto*) y está diseñado para transformar al estudiante de un usuario que ejecuta órdenes reactivas a un científico computacional capaz de diseñar procedimientos automáticos, robustos y verificables:

- **Fundamentos de Bash y sintaxis segura:** anatomía de órdenes, variables, caracteres especiales, comillas y aritmética entera.
- **Flujos y composición:** los tres canales estándar de Unix (`stdin`, `stdout`, `stderr`), operadores de redirección y la composición modular mediante tuberías (`|`).
- **De la orden al script:** cómo estructurar la lógica previa en pseudocódigo, empaquetar instrucciones en un script ejecutable, depurar con `bash -n` y blindar su ejecución con `set -euo pipefail`.
- **Estructuras de control y modularidad:** condicionales (`if/elif/else`, `[[ ]]`, `(( ))`), bucles (`for`, `while`, `read`, `break`, `continue`), funciones con ámbito local y operadores nativos de manipulación de cadenas.
- **La tríada Unix de procesamiento de texto:** el formato biológico *FASTA*, búsqueda con `grep` y expresiones regulares extendidas (*ERE*), transformación de secuencias con `sed` y análisis estructurado por columnas con `awk`.

Al finalizar el capítulo, se abrirá formalmente el puente hacia la algoritmia y el análisis del coste computacional, tema central del capítulo TC-CH03.

2 Fundamentos de Bash: órdenes, opciones y variables

2.1 Anatomía de una orden

Toda orden en la Shell de Bash se descompone en tres componentes fundamentales: `nombre_de_orden` [`opciones`] [`argumentos`]. El nombre indica el ejecutable que se desea lanzar; las opciones modifican el comportamiento del programa; y los argumentos representan los datos o rutas sobre los que actuará.

Las opciones suelen presentarse en dos formatos: **forma corta** con un único guion (como `-l` o `-h`) y **forma larga** con dos guiones (como `--all` o `--human-readable`). Cuando varias opciones cortas no requieren argumento propio, pueden combinarse tras un único guion (`-lh` equivale exactamente a `-l -h`).

Una orden de Bash se descompone en nombre de la orden, opciones (forma corta con un guion o larga con dos guiones) y argumentos; el orden relativo de las opciones entre sí normalmente no importa, pero su posición respecto a los argumentos sí puede importar según la orden.

Varias opciones cortas sin argumento propio pueden combinarse tras un único guion: `wc -l -w` y `wc -lw` piden el mismo resultado a `wc`.

Se reproduce con `verification/chapter_results.sh combined_short_options`.

2.2 Variables: asignar y recuperar un valor

Una variable es un contenedor con nombre que almacena información en la memoria de la Shell durante la sesión. En Bash, la asignación de una variable se escribe con la sintaxis `NOMBRE=valor`, con una condición indispensable: **no debe haber ningún espacio alrededor del signo igual**. Si se introduce un espacio (`NOMBRE = valor`), Bash interpretará que `NOMBRE` es un comando y que `=` y `valor` son sus argumentos, resultando en un fallo inmediato.

Para recuperar el valor almacenado en una variable se antepone el símbolo `$`: `$NOMBRE`. Cuando el nombre de la variable está adyacente a otros caracteres de texto que podrían confundir al analizador sintáctico, es obligatorio y una excelente práctica de ingeniería delimitar el nombre entre llaves: `${NOMBRE}`.

`NOMBRE=valor` asigna sin error; `NOMBRE = valor` (con espacios) no asigna la variable y falla o hace otra cosa.

Se reproduce con `verification/chapter_results.sh variable_assignment_spacing`.

`${NOMBRE}_sufijo` recupera `NOMBRE` y añade el sufijo literal; `$NOMBRE_sufijo` intenta recuperar una variable distinta que normalmente no existe y se expande a vacío.

Se reproduce con `verification/chapter_results.sh brace_disambiguation`.

2.3 Sustitución de órdenes, subshells y aritmética

A menudo necesitamos capturar el resultado emitido por una orden para guardarlo en una variable o pasarlo como argumento a otro programa. La sintaxis moderna y anidable de **sustitución de órdenes** es `$(orden)`. Bash ejecuta `orden`, captura su salida estándar (`stdout`), elimina el salto de línea final y sustituye la expresión por ese contenido.

Por otro lado, los paréntesis simples `( ... )` ejecutan el bloque de órdenes en un entorno secundario aislado o **subshell**. Cualquier cambio en variables o en el directorio de trabajo dentro del subshell se descarta al finalizar, sin alterar la sesión principal.

Para realizar cálculos matemáticos enteros, Bash proporciona la **expansión aritmética** mediante la sintaxis `$( ( expresión ) )`. Es fundamental recordar que la aritmética nativa de Bash es estrictamente **entera**: la división trunca la parte decimal hacia cero, por lo que `$( ( 7 / 2 ) )` da como resultado 3 y no 3.5.

`$(orden)` se sustituye por el `stdout` de `orden` sin el salto de línea final; el resultado puede asignarse a una variable o usarse directamente como argumento de otra orden.

Se reproduce con `verification/chapter_results.sh command_substitution`.

`$( ( 7 / 2 ) )` vale 3, no 3.5: la aritmética de `$( ( ) )` es entera y trunca hacia cero, no redondea.

Se reproduce con `verification/chapter_results.sh arithmetic_expansion`.

2.4 Separadores de línea y control de flujo

Además del salto de línea convencional, Bash reconoce operadores especiales para encadenar múltiples órdenes en una única línea con semántica de control:

- `;` (punto y coma): ejecuta la primera orden y, cuando termina, ejecuta la siguiente de forma incondicional.
- `&&` (Y lógico): ejecuta la segunda orden **únicamente si la primera terminó con éxito** (código de salida 0).
- `||` (O lógico): ejecuta la segunda orden **únicamente si la primera falló** (código de salida distinto de 0).
- `&` (ampersand simple): lanza la orden en segundo plano (*background*) y devuelve el control a la Shell de inmediato.

`cmdA && cmdB` ejecuta `cmdB` solo si `cmdA` termina con estado 0; `cmdA || cmdB` ejecuta `cmdB` solo si `cmdA` termina con estado distinto de cero; `;` encadena sin mirar el resultado; `&` lanza en segundo plano.

Se reproduce con `verification/chapter_results.sh line_separators`.

3 Caracteres especiales y comillas

En la Shell de Bash, ciertos caracteres tienen un significado especial y se denominan **metacaracteres**. La Shell interpreta estos caracteres antes de entregar los argumentos al programa ejecutable. Dominar el uso de las comillas es la habilidad más importante para escribir código seguro y predecible.

Carácter	Significado en Bash	Prevención
\$	Expansión de variables y sustituciones	'\$VAR' o \ \$VAR
* ? []	Coincidencia de nombres de fichero (<i>globbing</i>)	Entrecomillar o \
~	Expansión de directorio personal (<i>home</i>)	Entrecomillar
#	Inicio de comentario	Entrecomillar
; &	Separadores de órdenes y tuberías	Entrecomillar
espacio	Separador de palabras y argumentos	Comillas dobles " . . . "
\	Carácter de escape: quita significado al siguiente	\\
'	Comillas simples: suprime toda expansión	Cerrar comilla
"	Comillas dobles: permite \$ y \$(. . .), pero bloquea división	Cerrar comilla

Los metacaracteres de esta tabla son el conjunto mínimo necesario para leer y escribir con seguridad cualquier script de este capítulo.

La regla general de las comillas en Bash es sencilla pero crucial:

1. **Comillas simples** (' . . . '): proporcionan literalidad absoluta. Nada dentro de comillas simples se expande: ni variables ('\$VAR'), ni sustituciones ('\$ (pwd) '), ni comodines ('*.fasta'). Es el escudo más fuerte para preservar texto intacto.
2. **Barra invertida** (\): escapa un único carácter inmediatamente posterior, anulando su significado especial.
3. **Comillas dobles**: permiten la expansión de variables y la sustitución de órdenes, pero **bloquean la división en palabras** (*word splitting*) y la **expansión de rutas** (*globbing*). Es el entrecomillado que quiere casi siempre.

Dentro de comillas simples ninguna expansión ocurre; el texto se pasa literalmente carácter a carácter, incluido el signo dólar o el asterisco.

Se reproduce con `verification/chapter_results.sh single_quote_literal`.

Una barra invertida antes de un único carácter le quita su significado especial para esa aparición; \ \$VAR imprime literalmente el signo dólar seguido del nombre.

Se reproduce con `verification/chapter_results.sh backslash_escape`.

Dentro de comillas dobles sí se expanden \$variable y \$(orden), pero el resultado no se somete a división de palabras ni a expansión de *glob*.

Se reproduce con `verification/chapter_results.sh double_quote_expands_blocks_split`.

4 Flujos estándar: stdin, stdout, stderr

En la arquitectura de Unix, todo proceso en ejecución nace con tres canales de comunicación unidireccionales abiertos de forma predeterminada, identificados por un número entero o **descriptor de archivo**:

1. **Entrada estándar (stdin, descriptor 0)**: canal por el que el programa lee sus datos de entrada. Por defecto, está conectado al teclado.
2. **Salida estándar (stdout, descriptor 1)**: canal por el que el programa emite sus resultados normales de procesamiento. Por defecto, está conectado a la pantalla del terminal.
3. **Error estándar (stderr, descriptor 2)**: canal por el que el programa emite avisos, diagnósticos y mensajes de error. Aunque por defecto también se muestra en la pantalla, es un canal completamente independiente

de stdout.

Esta separación conceptual es un principio maestro de ingeniería: permite guardar o canalizar los datos biológicos válidos emitidos por stdout sin que queden contaminados por advertencias o diagnósticos de depuración emitidos por stderr.

stdin, stdout y stderr son tres flujos independientes de un proceso; separarlos permite conservar un resultado mientras los avisos se descartan o se inspeccionan por separado.

Se reproduce con `verification/chapter_results.sh separate_stderr`.

5 Redirección: escribir, añadir, leer y separar errores

Los operadores de redirección permiten alterar el destino o el origen de los flujos estándar, conectándolos a archivos en disco en lugar de la terminal.

Operador	Significado y comportamiento	Ejemplo
>	Redirige stdout a fichero, truncando previo	<code>orden > salida.txt</code>
>>	Redirige stdout a fichero, añadiendo al final	<code>orden >> salida.txt</code>
<	Redirige fichero hacia stdin	<code>orden < entrada.txt</code>
2>	Redirige solo stderr a fichero	<code>orden 2> errores.log</code>
2>>	Redirige stderr añadiendo al final	<code>orden 2>> errores.log</code>
&>	Redirige ambos (stdout y stderr)	<code>orden &> todo.log</code>

>, >>, <, 2>/2>> y &> son los operadores de redirección documentados en el manual de Bash para este capítulo.

Advertencia sobre la truncación inmediata. El operador > trunca y vacía el archivo de destino en el mismo instante en que la Shell interpreta la orden, **antes** de que el comando empiece a ejecutarse. Si un script intenta leer y escribir sobre el mismo archivo a la vez (`sort muestra.txt > muestra.txt`), el archivo quedará completamente vacío antes de que sort pueda leer una sola línea, perdiéndose los datos de forma irreversible.

`orden > fichero` trunca fichero antes de escribir, incluso si la orden falla y no produce salida; el contenido previo se pierde.

Se reproduce con `verification/chapter_results.sh truncate_on_redirect`.

2> redirige únicamente stderr; una orden ejecutada con salida y error redirigidos por separado escribe su resultado y sus diagnósticos en ficheros distintos.

Se reproduce con `verification/chapter_results.sh separate_stderr`.

>> añade al final del destino sin truncar el contenido previo, a diferencia de >; dos escrituras sucesivas conservan ambos bloques de datos.

Se reproduce con `verification/chapter_results.sh append_does_not_truncate`.

6 Tuberías: componer programas sin que se conozcan entre sí

El operador de tubería (`|`, *pipe*) es la encarnación más elegante de la filosofía Unix: conecta directamente la salida estándar (stdout) de un programa con la entrada estándar (stdin) del siguiente.

En `a | b`, el stdout de a se convierte en el stdin de b; ni a ni b necesitan saber que su entrada o salida proviene de una tubería.

6.1 El peligro del estado de salida en tuberías: pipefail

Por defecto, en una tubería compuesta (`cmd1 | cmd2 | cmd3`), la Shell asigna a la variable especial `$?` el código de salida del **último comando ejecutado** (`cmd3`). Si el primer comando (`cmd1`) falla catastróficamente (por ejemplo, porque el archivo de datos no existe), pero `cmd3` termina correctamente recibiendo una entrada vacía, la Shell reportará que la tubería tuvo éxito (`$? = 0`).



Ventaja Arquitectónica de la Tubería Unix (|):

- Los datos fluyen de proceso a proceso a través de buffers de memoria en el núcleo (kernel de ~64 KB).
- Cero archivos temporales en disco: se pueden procesar cientos de gigabytes consumiendo solo unos megabytes de RAM.
- Paralelismo implícito: los tres procesos se ejecutan simultáneamente en núcleos de CPU separados.

Árbol Sintáctico de Expresiones Regulares (POSIX ERE):

- Anclajes: `^` (Inicio de línea) | `$` (Fin de línea)
- Clases: `[ACGT]` (Un nucleótido) | `[^N]` (Cualquier base excepto N)
- Cuantificadores: `+` (1 o más) | `*` (0 o más) | `{min,max}` (Rango)

Figura 1: Flujograma del modelo de tuberías Unix (|) conectando grep, sed y awk en memoria, junto con el árbol sintáctico de expresiones regulares POSIX ERE. Figura original adaptada de las presentaciones docentes.

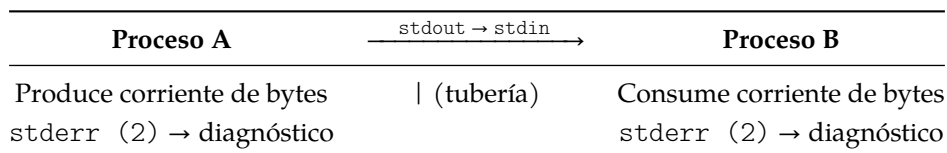


Figura 2: Esquema de una tubería Unix. Los programas no se conocen entre sí; solo el flujo de texto que cruza el pipe los acopla.

En bioinformática, este comportamiento por defecto puede arruinar un análisis completo sin dejar rastro de error. Para solucionar esto, activamos la opción `set -o pipefail`. Con `pipefail`, el estado de salida de la tubería será el del comando que haya fallado más a la derecha, de modo que un fallo en una etapa temprana ya no pasa desapercibido.

Atención. `pipefail` tiene un efecto secundario que conviene conocer antes de tropezar con él. Si una etapa posterior cierra la tubería antes de tiempo, como hace `... | head -n 5` en cuanto tiene sus cinco líneas, la etapa anterior recibe la señal `SIGPIPE` y termina con estado 141. Con `set -e` activo, el script muere aunque el resultado sea exactamente el que usted quería. Evite `head` en tuberías bajo `set -e`, o justifique con un comentario el `|| true` que añada.

Sin `set -o pipefail`, el estado de salida de una tubería de varias etapas refleja solo el del último comando; el fallo del primero pasa inadvertido.

Se reproduce con `verification/chapter_results.sh pipeline_exit_status`.

7 De la orden interactiva al script

7.1 De la idea al pseudocódigo

Escribir un script no consiste en teclear órdenes apresuradas en un archivo. El desarrollo profesional comienza siempre diseñando el procedimiento en **pseudocódigo**: una descripción lógica y estructurada en lenguaje natural, libre de particularidades de sintaxis, que descompone el problema biológico en pasos elementales:

```

Para cada archivo que termine en .fasta dentro de datos/corpus/:
  Comprobar si el archivo contiene datos regulares
  Contar el numero de registros (lineas que empiezan por >)
  Si tiene mas de 100 registros:
    Anotar el archivo en la lista de alta densidad
  Si no:
  
```

```
Anotar el archivo en la lista estandar
Fin del bucle
```

Escribir primero el procedimiento en pseudocódigo —lenguaje claro, sin sintaxis exacta— antes de traducirlo a Bash reduce errores de diseño y clarifica el flujo de datos.

7.2 El Shebang y las opciones de seguridad (`set -euo pipefail`)

Un script de Bash es un archivo de texto con permisos de ejecución (`chmod +x script.sh`). La primera línea debe contener el **shebang**: `#!/usr/bin/env bash`, que indica al sistema operativo qué intérprete debe ejecutar el archivo.

Para que los fallos sean visibles en vez de silenciosos, los scripts de este curso empiezan siempre por el mismo par de líneas [4]:

```
#!/usr/bin/env bash
set -euo pipefail
```

- `set -e`: termina el script si falla una orden simple. No actúa sobre la condición de un `if` o un `while`, ni sobre los miembros izquierdos de `&&` y `||`, ni sobre las etapas intermedias de una tubería mientras no active también `pipefail`. Es una red de seguridad amplia, no una garantía.
- `set -u`: termina con error si se intenta leer una variable no definida, evitando que una errata tipográfica se convierta silenciosamente en una cadena vacía.
- `set -o pipefail`: propaga los fallos ocurridos en cualquier eslabón de una tubería.

Límites y tratamiento explícito de excepciones.

Esta cabecera no demuestra que el script sea matemáticamente correcto ni sustituye la validación explícita de entradas y precondiciones. Cuando un comando puede devolver código no cero como comportamiento esperado (por ejemplo, cuando `grep` no encuentra coincidencias en una búsqueda condicional), el script debe gestionar ese estado explícitamente mediante una estructura condicional (`if`) o una alternativa controlada (`|| true`), en vez de delegar pasivamente la decisión en `set -e`.

Antes de ejecutar un script, podemos verificar si contiene errores de sintaxis sin lanzarlo realmente utilizando la opción de prueba en seco `bash -n script.sh`. Para seguir la traza completa de ejecución paso a paso, usamos `bash -x script.sh`.

`bash -n script.sh` comprueba la sintaxis sin ejecutar el script; detecta errores sintácticos, pero no de lógica ni de datos.

Se reproduce con `verification/chapter_results.sh syntax_check`.

`set -u` hace que el script termine con error al hacer referencia a una variable no definida, en vez de sustituirla silenciosamente por una cadena vacía.

Se reproduce con `verification/chapter_results.sh set_u_catches_typo`.

`set -o pipefail` hace que el estado de salida de una tubería sea distinto de cero si cualquiera de sus etapas falla.

Se reproduce con `verification/chapter_results.sh pipeline_exit_status`.

8 Variables, comillas y argumentos posicionales

Para que un script sea verdaderamente reutilizable, debe aceptar argumentos por línea de comandos. Dentro del script, estos argumentos se reciben en variables especiales denominadas **parámetros posicionales**:

- `$0`: nombre o ruta con la que se invocó el propio script.
- `$1`, `$2`, ..., `$9`: primer, segundo y sucesivos argumentos posicionales.

- `$#` : número total de argumentos recibidos.
- `"$@"` : expande todos los argumentos posicionales manteniendo cada uno como una palabra separada e individual, incluso si contienen espacios internos. Es la forma correcta de pasar los argumentos uno a uno; `"$*"` sirve para lo contrario, unirlos todos en una sola cadena.

Dentro de un script, `$0` es el nombre del propio script, `$1` es el primer argumento, `$#` es el número total de argumentos y `"$@"` expande todos los argumentos preservando sus límites.

Se reproduce con `verification/chapter_results.sh positional_args` .

Una variable sin comillas que representa una ruta se somete a división de palabras y expansión de *glob*; si contiene espacios o comodines, la orden recibe más o menos argumentos de los previstos.

Se reproduce con `verification/chapter_results.sh quoting_matters` .

9 Aplicación: proteger una operación destructiva

Una trampa clásica en scripting ocurre al combinar variables con operaciones de borrado (`rm -rf`). Consideremos el siguiente caso:

```
DIRECTORIO_TEMP=""
# ERROR PELIGROSO:
# rm -rf "$DIRECTORIO_TEMP"/*    <- Si la variable esta vacia, se expande a rm -rf /*
```

Aunque tengamos activado `set -u` , esta opción solo detecta variables **no declaradas**. Una variable declarada pero asignada a la cadena vacía (`VAR=""`) pasa `set -u` sin emitir ningún error. Al concatenarse con el comodín fuera de las comillas (`"$DIRECTORIO_TEMP"/*`), la expresión se expande a `/*` (la raíz del sistema de archivos), con consecuencias catastróficas. Si el asterisco estuviera dentro de las comillas (`"$DIR/*"`), no expandiría comodines sino que buscaría un nombre literal con asterisco.

Para evitar desastres, seguimos el protocolo preventivo de comprobación:

1. Validar con `[[ -n "$VAR" ]]` que la variable contenga una cadena no vacía.
2. Validar con `[[ -d "$VAR" ]]` que el directorio exista en disco.
3. Realizar siempre una inspección inocua antes de cualquier acción, para ver exactamente en qué se expande la ruta:

```
printf '<%s>\n' "$DIRECTORIO_TEMP"/*
```

`set -u` detecta una variable *no definida*, pero no detecta una variable definida y vacía (`VAR=""`); `rm -rf "$VAR/*"` pasa sin error y resulta destructivo sobre un ámbito indeseado.

Se reproduce con `verification/chapter_results.sh empty_var_passes_set_u` .

El protocolo de TC-CH01 se completa aquí con un paso adicional específico de scripts parametrizados: mostrar la expansión exacta con un comando inofensivo como `printf` antes de ejecutar cualquier acción destructiva.

Se reproduce con `verification/chapter_results.sh destructive_dry_run` .

9.1 Escribir el script: un editor mínimo

ESTUDIO Un script es un fichero de texto, así que necesita un editor dentro del terminal. Con dos basta para este curso.

`nano` es el más sencillo: se abre con `nano script.sh` , se escribe directamente y las órdenes aparecen abajo, donde el símbolo `^` significa la tecla Control. Guardar es Control-O y salir Control-X.

`vim` es más incómodo al principio y está en cualquier servidor, incluso en los que no tienen `nano` instalado, así que conviene conocer el mínimo para no quedarse atrapado: se abre con `vim script.sh` , arranca en modo orden, se pasa a modo inserción con `i` , se vuelve al modo orden con Escape, y desde ahí `:wq` guarda y sale, `:q!` sale descartando los cambios.

Concepto. Da igual cuál elija. Lo que no da igual es que el script quede guardado en el proyecto y no en el historial de la terminal: el fichero se puede leer, versionar y corregir; la orden tecleada, no.

9.2 Cuatro formas de ejecutar un script, y en qué se diferencian

- `bash script.sh`: lanza un intérprete nuevo que lee el fichero. No hace falta permiso de ejecución, y la línea `#!` se ignora.
- `./script.sh`: el sistema lee la línea `#!` para saber con qué intérprete ejecutarlo. Necesita permiso de ejecución.
- `script.sh`, sin ruta: solo funciona si el directorio está en `PATH`. El directorio actual no lo está, y por eso el punto y la barra del caso anterior no son un adorno.
- `source script.sh` o `. script.sh`: no lanza un proceso nuevo; ejecuta las órdenes en la sesión actual. Las variables que defina el script permanecen después. Es la forma de cargar un fichero de configuración, y una fuente de sorpresas si lo usa por costumbre: un `exit` dentro del script cierra su terminal.

10 Condicionales

10.1 `[]` frente a `[[ ]]`

Para evaluar condiciones en Bash disponemos del comando clásico `[]` (o `test`) y de la construcción moderna extendida `[[ ]]`. En scripts de Bash conviene **usar** `[[ ]]`, que evita errores de división de palabras y permite operadores lógicos avanzados como `&&`, `||` y coincidencia de patrones con expresiones regulares (`=~`). El corchete simple no es un resto del pasado: sigue siendo necesario cuando el script debe correr bajo `sh` POSIX, donde `[[ ]]` no existe. Si su cabecera dice `bash`, use el doble; si dice `sh`, no puede.

Aspecto	<code>[]</code> (clásico / POSIX)	<code>[[]]</code> (Bash moderno)
Comillas en variables	Obligatorias siempre (" <code>\$var</code> ")	Muy recomendadas por estilo
División de palabras	Sí, falla si la variable contiene espacios	No se produce dentro de <code>[[]]</code>
Operadores lógicos	<code>-a</code> (AND), <code>-o</code> (OR)	<code>&&</code> (AND), <code> </code> (OR)
Coincidencia de patrones	No	Sí (<code>== *.txt</code> , <code>= ^[0-9]+\$</code>)

`[]` y `[[ ]]` difieren en la necesidad de comillas y en los operadores lógicos que aceptan (`-a/-o` frente a `&&/||`); `[[ ]]` es la forma estándar en Bash.

10.2 Álgebra booleana y operadores de prueba

Operador	Significado de la condición
<code>[[-f "\$fichero"]]</code>	Cierto si el fichero existe y es un fichero regular
<code>[[-d "\$dir"]]</code>	Cierto si la ruta existe y es un directorio
<code>[[-s "\$fichero"]]</code>	Cierto si el fichero existe y su tamaño es mayor que cero
<code>[[-r "\$fichero"]]</code>	Cierto si el fichero tiene permiso de lectura
<code>[[-z "\$cadena"]]</code>	Cierto si la cadena de texto está vacía (longitud 0)
<code>[[-n "\$cadena"]]</code>	Cierto si la cadena de texto no está vacía
<code>[["\$a" == "\$b"]]</code>	Comparación de igualdad textual
<code>[["\$a" != "\$b"]]</code>	Comparación de desigualdad textual
<code>((a == b))</code>	Comparación de igualdad numérica
<code>((a > b))</code>	Comparación de desigualdad numérica (mayor que)
<code>[[-e "\$ruta"]]</code>	Cierto si la ruta existe, sea del tipo que sea

Operador	Significado de la condición
<code>[[-L "\$ruta"]]</code>	Cierto si la ruta es un enlace simbólico
<code>[["\$a" -nt "\$b"]]</code>	Cierto si a se modificó más recientemente que b
<code>[["\$m" -eq "\$n"]]</code>	Igualdad entera dentro de corchete simple; sus parientes son <code>-ne</code> , <code>-gt</code> , <code>-ge</code> , <code>-lt</code> y <code>-le</code>

Atención. Los operadores enteros y los textuales no son intercambiables. `[[ "10" > "9" ]]` es *falso*, porque compara texto y en el orden alfabético el uno va antes que el nueve; `(( 10 > 9 ))` es cierto. Cuando compare tamaños, conteos o posiciones, use la forma aritmética. Y si necesita el «uno u otro pero no los dos», el o exclusivo, no existe operador propio: se escribe como `[[ $a != $b ]]` sobre dos condiciones booleanas ya evaluadas.

Los operadores de cadena, enteros, ficheros y lógicos de esta tabla son el conjunto mínimo necesario para evaluar cualquier condición en los scripts de este capítulo.

`[[ -f "$ruta" ]]` comprueba si `$ruta` es un fichero regular existente; `[[ -z "$var" ]]` comprueba si `var` está vacía; `(( a > b ))` compara a y b como números, no como texto.

Se reproduce con `verification/chapter_results.sh test_operators_string_file_numeric`.

10.3 if/elif/else

```
if [[ condicion1 ]]; then
    comando1
elif [[ condicion2 ]]; then
    comando2
else
    comando3
fi
```

`if/elif/else/fi` evalúa condiciones en orden y ejecuta el primer bloque cuya condición sea verdadera; `else` se ejecuta solo si ninguna lo fue.

`(( a > b ))` compara a y b como números; `[[ "$a" > "$b" ]]` los compara como cadenas de texto y puede dar un resultado distinto (`"9" > "10"` es cierto lexicográficamente aunque falso numéricamente).

Se reproduce con `verification/chapter_results.sh numeric_vs_lexicographic`.

11 Bucles

11.1 for: iterar un número conocido de veces

El bucle `for` permite repetir un bloque de instrucciones sobre una colección de elementos o mediante un índice numérico estilo C:

```
for i in 1 2 3; do echo "$i"; done

for ((i=1; i<=3; i+=1)); do echo "$i"; done
```

El bucle `for` sobre una lista y el bucle `for` con inicialización, condición y paso son dos sintaxis equivalentes en Bash; la segunda es más cómoda cuando se itera con un contador numérico.

Se reproduce con `verification/chapter_results.sh for_loop_both_styles`.

11.2 while y lectura línea a línea

El bucle `while` repite instrucciones mientras una condición sea cierta. Combinado con el comando `read`, constituye el patrón más eficiente y seguro para procesar archivos de texto gigantescos línea a línea:

```
while IFS= read -r linea; do
    echo "Procesando: $linea"
done < archivo.txt
```

- IFS=: suprime el separador de campos por defecto para no recortar espacios en blanco al inicio o final de la línea.
- -r: desactiva la interpretación de la barra invertida como escape.

while IFS= read -r linea; do ... done < fichero procesa fichero línea a línea; IFS= evita recortar espacios en los extremos y -r evita que la barra invertida se trate como carácter de escape dentro de la línea leída.

Se reproduce con `verification/chapter_results.sh while_read_counts_lines`.

11.3 break y continue

Para modular el comportamiento de los bucles:

- break: interrumpe y finaliza el bucle de inmediato.
- continue: interrumpe la iteración actual y salta directamente a la siguiente.

Dentro de un for, break detiene el bucle antes de procesar los elementos restantes; continue omite el resto del cuerpo para el elemento actual pero sigue con el siguiente.

Se reproduce con `verification/chapter_results.sh break_and_continue`.

11.4 Dos formas de while que aparecen constantemente

La primera es el bucle sin condición, que se detiene desde dentro. Se usa cuando la condición de salida solo se conoce a mitad de la iteración:

```
while true; do
    read -r -p "Escriba «fin» para terminar: " respuesta
    [ "$respuesta" = fin ] && break
    echo "recibido: $respuesta"
done
```

La segunda reparte cada línea en varios campos a la vez. read corta por los separadores de IFS y asigna en orden; la última variable recoge todo lo que sobra, lo que resulta muy práctico cuando el último campo lleva espacios:

```
while read -r muestra condicion descripcion; do
    echo "$muestra ($condicion): $descripcion"
done < muestras.tsv
```

Atención. La opción -r no es opcional en la práctica. Sin ella, read interpreta la barra invertida como carácter de escape y le destroza cualquier ruta de Windows o cualquier secuencia que la contenga.

12 Funciones

Una función agrupa un conjunto de instrucciones bajo un nombre descriptivo para su reutilización:

```
saludar() {
    local nombre="$1"
    echo "Hola $nombre"
}
saludar "Ana"
resultado="$(saludar "Ana")"
```

Es fundamental distinguir dos conceptos:

1. **return N**: solo devuelve un **código de estado numérico** (de 0 a 255) accesible mediante `$?`. No devuelve texto.
2. **echo o printf**: emite datos a la salida estándar, los cuales pueden capturarse en variables mediante `$(funcion)`.

Una función de Bash se define con `nombre() { ... }` y se invoca como una orden; sus argumentos se leen con `$1, $2`, etc.

Se reproduce con `verification/chapter_results.sh function_define_and_call`.

`return N` termina una función con estado de salida `N` (accesible en `$?`); para devolver datos textuales se imprime con `echo/printf` y se captura con `$(...)`.

Se reproduce con `verification/chapter_results.sh function_return_vs_output`.

12.1 Buenas prácticas al escribir funciones

Bash admite dos sintaxis equivalentes, `nombre() { ... }` y `function nombre { ... }`. La primera es portable a `sh`; la segunda solo funciona en Bash. Elija una y manténgala en todo el script.

- Declare como `local` toda variable interna. Sin `local`, la variable es global y una función puede pisar el valor de otra sin que nada avise.
- Valide los argumentos en la primera línea del cuerpo, y termine con un mensaje claro si faltan: `[ $# -eq 2 ] || { echo "uso: ..." >&2; return 2; }`.
- Escriba en la salida estándar lo que la función *produce*, y en la de error lo que *informa*. Así quien la llame puede capturar el resultado con `$(funcion)` sin que se le mezclen los avisos.
- Use `return` para el estado, no para el resultado: `return` solo admite un número entre 0 y 255.
- Ponga a cada función un comentario de una línea que diga qué recibe y qué devuelve. Dentro de tres meses será usted quien lo lea.

13 Arrays y expansión de llaves

13.1 Arrays y recorrido seguro de ficheros

Al recorrer colecciones de archivos con comodines (`*.fasta`), si no hay ningún archivo coincidente, Bash deja por defecto el patrón literal sin expandir (`"*.fasta"`). Para evitar que el bucle intente procesar un fichero inexistente con ese nombre, activamos `shopt -s nullglob`:

```
shopt -s nullglob
ficheros=(datos/*.fasta)
if [[ ${#ficheros[@]} -eq 0 ]]; then
    echo "Aviso: no se encontraron archivos FASTA"
fi
```

Con `shopt -s nullglob`, un array construido como `f=(*.fasta)` queda vacío si no hay coincidencias en el directorio, en vez de contener el patrón literal sin expandir.

Se reproduce con `verification/chapter_results.sh nullglob_empty_dir`.

13.2 Expansión de llaves

La expansión de llaves genera combinaciones y secuencias de texto de forma puramente sintáctica antes de ejecutar la orden:

- Rangos numéricos o de caracteres: `{1..5}` genera `1 2 3 4 5`.

- Listas de alternativas: `muestra_{A,B,C}.txt` genera los tres nombres `muestra_A.txt`, `muestra_B.txt` y `muestra_C.txt`.
- Rangos con paso: `{1..10..2}` genera 1 3 5 7 9, y `{0..100..25}` sirve para recorrer umbrales sin escribirlos.

Atención. La expansión de llaves ocurre *antes* que la de comodines y no consulta el disco: `muestra_{A,B,C}.txt` genera los tres nombres existan o no. Si lo que quiere es «los ficheros que hay», use un comodín; si quiere «los nombres que deberían estar», use llaves y compruebe después cuáles faltan.

`{1..5}` se expande a 1 2 3 4 5 antes de que la Shell ejecute la orden; `{A,B}.txt` genera `A.txt` `B.txt` sin consultar el sistema de ficheros.

Se reproduce con `verification/chapter_results.sh brace_expansion_range_and_list`.

14 Operadores nativos de cadena

Bash incluye potentes operadores integrados de manipulación de cadenas que evitan lanzar procesos externos:

- **Extracción de subcadenas:** `${var:pos:long}` extrae `long` caracteres a partir de la posición `pos`.
- **Sustitución de patrones:** con una barra, `${var/antiguo/nuevo}`, sustituye la primera coincidencia; con dos barras, todas.
- **Eliminación de prefijos:** `${var#prefijo}` elimina el prefijo coincidente más corto; `${var##prefijo}` el más largo.
- **Eliminación de sufijos:** `${var%sufijo}` elimina el sufijo más corto (ideal para quitar extensiones de fichero: `${fichero%.fasta}`); `${var%%sufijo}` elimina el más largo, que es lo que hace falta con nombres como `muestra.trim.sorted.bam` cuando quiere quedarse solo con `muestra`.
- **Último carácter:** `${var: -1}` devuelve el último carácter. Los paréntesis no son decoración: sin ellos, los dos puntos seguidos de un signo menos significan otra cosa.

REFERENCIA Tres construcciones más que aparecerán en scripts ajenos. `$RANDOM` entrega un entero pseudoaleatorio distinto en cada lectura, útil para nombrar un temporal y desaconsejable para muestrear datos, porque no se puede fijar su semilla. `declare -i contador` marca una variable como entera, de modo que `contador=contador+1` funciona sin `$( ( ))`. Y `${!muestra_*}` lista los nombres de todas las variables que empiezan por `muestra_`, que es la forma de recorrer un conjunto de parámetros declarados por separado.

`${var:0:3}` extrae los tres primeros caracteres de `var`; `${var/e/E}` sustituye la primera coincidencia y `${var//e/E}` sustituye todas.

Se reproduce con `verification/chapter_results.sh string_slice_and_substitute`.

`${var#prefijo}` elimina prefijo del inicio de la variable; `${var%sufijo}` elimina sufijo del final (útil para extensiones de fichero).

Se reproduce con `verification/chapter_results.sh string_prefix_suffix_removal`.

15 Hito de consolidación: del programa robusto al texto biológico

Hito 1 • Dominio de la construcción de programas robustos en Bash.

Hasta este punto del capítulo, hemos formalizado las cuatro capacidades fundamentales de la programación en Shell:

1. **Recepción segura de datos:** Manejo de argumentos posicionales con `"$@"` y protección contra separación de palabras y variables vacías.
2. **Estructuras de decisión:** Condicionales robustos con `[[ ... ]]` y evaluación lógica explícita.
3. **Control de flujo y repetición:** Bucles `for` sobre listas y archivos con `nullglob`, y bucles `while read` para flujos línea a línea.

4. **Modularidad:** Definición y llamada a funciones parametrizadas que devuelven resultados limpios.

A partir de la siguiente sección, aplicamos estas herramientas de automatización a nuestro objeto biológico principal: el procesamiento masivo de texto y secuencias genómicas. Para abordar cualquier fichero estructurado, distinguiremos tres operaciones complementarias:

- **Filtrar con grep:** Selecciona líneas enteras que coinciden con un patrón sin alterar su contenido.
- **Transformar con sed:** Modifica, sustituye o recorta el texto línea a línea de forma programática.
- **Procesar con awk:** Descompone cada línea en columnas/campos estructurados, acumulando estado y calculando estadísticas.

Comprobación de integración rápida.

Antes de continuar, prediga qué construcción utilizaría en cada caso: (a) ¿Cómo recorrer de forma segura todos los ficheros *.fasta de una carpeta sin fallar si no hay ninguno? → Con un array y la opción shopt -s nullglob. (b) ¿Cómo seleccionar exclusivamente las líneas de cabecera en un archivo genómico? → Con la expresión regular ^> en grep. (c) ¿Cómo garantizar que un pipeline detecte si falló la primera etapa de una tubería? → Con la directiva set -o pipefail.

16 El formato FASTA como primer registro multilinea

El formato **FASTA** es el estándar universal en biología computacional para almacenar secuencias de nucleótidos o aminoácidos. Su estructura es estrictamente multilinea:

1. Una **línea de cabecera** que comienza obligatoriamente con el símbolo mayor que (>), seguido de un identificador y una descripción opcional.
2. Una o más **líneas de secuencia** que contienen las letras del alfabeto biológico (A, C, G, T, N, etc.).

Atención. Su pariente cercano, el formato FASTQ que entrega el secuenciador, no se procesa igual. Sus cabeceras empiezan por arroba, cada registro ocupa exactamente cuatro líneas, y la cuarta guarda las calidades, donde la arroba es un carácter de calidad perfectamente válido. Buscar líneas que empiecen por arroba en un FASTQ cuenta cabeceras y líneas de calidad, y devuelve un número inflado. Para contar registros en un FASTQ se dividen las líneas entre cuatro; para contarlos en un FASTA se cuentan las cabeceras.

Concepto. Los ficheros de secuencias suelen llegar comprimidos, con extensión .gz, y descomprimirlos para mirarlos ocupa espacio y tiempo. Las utilidades zcat, zgrep y zless leen el fichero comprimido y escriben el contenido descomprimido en la salida, sin dejar copia en disco: zcat muestras.fasta.gz | grep -c '^>' cuenta secuencias sin descomprimir nada.

```
>seq1 Secuencia humana de prueba
ATGCGATCGATCGATC
GATCGATCGA
>seq2 Secuencia control con bases indeterminadas
ATGCGTNNNNNNCCGT
```

Diferencia entre contar líneas y contar registros. Un archivo FASTA puede tener sus secuencias repartidas en varias líneas. Por tanto, contar líneas con wc -l **no** indica cuántas secuencias hay. Para contar registros biológicos reales debemos contar cuántas líneas comienzan por >, usando grep -c '^>' fichero.fasta.

Un registro FASTA consiste en una línea de cabecera que empieza por > seguida de una o más líneas de secuencia hasta la siguiente cabecera o el final del fichero.

grep -c '^>' fichero.fasta cuenta el número de **registros** del fichero, no su número de líneas.

Se reproduce con verification/chapter_results.sh fasta_header_count.

```
ALGORITMO: Parser_FASTA_Multilinea_Streaming
ENTRADA:
```

```

- Flujo_Texto: Stream secuencial de lineas de entrada (stdin o archivo)
SALIDA:
- Stream_Registros: Tuplas emitidas (Cabecera, Secuencia_Completa, Longitud, Porcentaje_GC)
COMPLEJIDAD: Tiempo  $O(N)$  donde  $N$  es el total de caracteres, Espacio  $O(L_{max})$  para el buffer

1. INICIALIZACION:
  Cabecera_Actual <- ""
  Buffer_Secuencia <- ""

2. BUCLE STREAMING:
  Para cada Linea en Flujo_Texto:
    Linea <- Eliminar_Espacios_Extremos(Linea)
    Si Linea es vacia entonces:
      Continuar al siguiente ciclo

    Si Linea comienza por '>' entonces:
      # Si teniamos un registro previo acumulado, lo procesamos y emitimos
      Si Cabecera_Actual != "" entonces:
        Longitud <- Longitud_Cadena(Buffer_Secuencia)
        Conteo_GC <- Contar_Ocurrencias(Buffer_Secuencia, {'G', 'C', 'g', 'c'})
        Si Longitud > 0 entonces:
          Porcentaje_GC <- (Conteo_GC / Longitud) * 100.0
        Sino:
          Porcentaje_GC <- INDEFINIDO
        AVISAR ("registro sin secuencia: ", Cabecera_Actual)
        EMITIR (Cabecera_Actual, Buffer_Secuencia, Longitud, Porcentaje_GC)

      # Inicializar nuevo registro
      Cabecera_Actual <- Linea
      Buffer_Secuencia <- ""
    Sino:
      # Acumular linea de nucleotidos/aminoacidos en el buffer
      Buffer_Secuencia <- Concatenar(Buffer_Secuencia, Linea)

3. TERMINACION (ULTIMO REGISTRO):
  Si Cabecera_Actual != "" y Buffer_Secuencia != "" entonces:
    Longitud <- Longitud_Cadena(Buffer_Secuencia)
    Conteo_GC <- Contar_Ocurrencias(Buffer_Secuencia, {'G', 'C', 'g', 'c'})
    Si Longitud > 0 entonces:
      Porcentaje_GC <- (Conteo_GC / Longitud) * 100.0
    Sino:
      Porcentaje_GC <- INDEFINIDO
    AVISAR ("registro sin secuencia: ", Cabecera_Actual)
    EMITIR (Cabecera_Actual, Buffer_Secuencia, Longitud, Porcentaje_GC)

```

17 Buscar con grep

grep (*Global Regular Expression Print*) es la herramienta habitual para buscar y filtrar patrones de texto en Unix.

Líneas coincidentes frente a ocurrencias totales. Si una línea de secuencia contiene tres repeticiones del motivo buscado, grep -c devolverá 1 (esa línea contiene al menos una coincidencia). Para contar las ocurrencias individuales exactas debemos extraer cada una con grep -o y contar con wc -l.

grep -c PATRÓN fichero cuenta **líneas** que contienen al menos una coincidencia, no el número total de coincidencias; una línea con dos coincidencias cuenta como una sola.

Se reproduce con verification/chapter_results.sh grep_count_vs_occurrences.

-n, -i, -v, -c, -o, -q, -r/-R y -l son las opciones de grep documentadas en el manual de GNU Grep para este capítulo. egrep es la forma histórica equivalente a grep -E.

17.1 Opciones avanzadas y control de contexto

Opción	Efecto	Ejemplo
-A n/-B n/-C n	líneas de contexto después/antes/ambos	grep -A2 '^>' f.fasta
-w	exige coincidencia de palabra completa	-w 'AT' no coincide con ATG
-x	exige que la línea entera coincida	grep -x 'ATGC' f.txt
-e PATRÓN (repetible)	añade un patrón más, en relación O	-e A -e T coincide con líneas que tengan A o T
-F	trata el patrón como cadena literal, no regex	un punto deja de ser comodín

La tabla de opciones extendidas de grep resume el control de contexto (-A/-B/-C), delimitación de palabra (-w), línea completa (-x), patrones múltiples (-e) y búsqueda literal rápida (-F).

grep -A n/-B n/-C n añaden a la salida las n líneas posteriores, anteriores o ambas alrededor de cada coincidencia.

Se reproduce con `verification/chapter_results.sh grep_context`.

grep -w 'AT' no coincide con una línea que contiene ATG, aunque sí coincide con una línea donde AT está rodeado de espacios o al final de línea.

Se reproduce con `verification/chapter_results.sh grep_word_boundary`.

grep -x PATRÓN solo cuenta como coincidencia una línea donde el patrón, anclado, cubre la línea completa.

Se reproduce con `verification/chapter_results.sh grep_exact_line`.

grep -e PATRON1 -e PATRON2 fichero es equivalente a grep -E 'PATRON1|PATRON2' fichero: ambas formas devuelven las líneas que coinciden con al menos uno de los dos patrones.

Se reproduce con `verification/chapter_results.sh grep_multiple_patterns`.

grep -F trata el patrón como cadena literal, no como expresión regular; un punto busca el carácter punto exacto y no actúa como comodín.

Se reproduce con `verification/chapter_results.sh grep_fixed_string`.

18 Expresiones regulares

18.1 Fundamento y sintaxis extendida (ERE)

Las expresiones regulares (*REGEX*) son un lenguaje formal para describir patrones en cadenas de texto, originadas en los trabajos sobre autómatas finitos del matemático Stephen Kleene (1951) e introducidas en Unix por Ken Thompson en el editor ed, hacia 1970, y en grep, en 1973.

Las expresiones regulares derivan del álgebra de autómatas finitos formalizada por Kleene (1951) e implementada en Unix por Thompson (1968, 1973).

Construcción	Significado	Ejemplo biológico
^	Ancla de inicio de línea	^> (cabeceras FASTA)
\$	Ancla de fin de línea	TAA\$ (codón de parada al final)
.	Cualquier carácter excepto salto de línea	A.T (A seguido de cualquier base y T)
[]	Clase de caracteres	[AG] (purinas)
[^]	Clase negada	[^ACGT] (bases no canónicas)
*	Cuantificador: 0 o más veces	N*
+	Cuantificador: 1 o más veces	N+ (islas de indeterminación)
?	Cuantificador: 0 o 1 vez (opcional)	AUG?
{n,m}	Entre n y m repeticiones	N{2,} (dos o más bases N)

Construcción	Significado	Ejemplo biológico
()	Grupo de captura	(CG){2,} (repetición del dinucleótido CG)
	Alternancia (O lógico)	TAA TAG TGA (codones de parada)

Atención. Un patrón como (CG){2,} encuentra dinucleótidos CG contiguos, y eso no es una isla CpG. Una isla CpG se define por densidad sobre una ventana, típicamente varios centenares de bases con un contenido GC alto y una razón entre CpG observados y esperados por encima de un umbral. Ninguna expresión regular cuenta densidades. Es un buen ejemplo de la frontera del capítulo: la expresión regular reconoce patrones de texto, no conceptos biológicos, y confundir las dos cosas produce anotaciones que parecen correctas.

Anclas, clases, cuantificadores (incluido ? para cero o una aparición), grupos y alternancia son las cinco construcciones mínimas de ERE para describir patrones sobre secuencias de texto.

Sobre data/tc02_caso_regex.fasta, el patrón N{2,} coincide con la línea de secuencia de caso_dos y no con la de caso_uno.

Se reproduce con `verification/chapter_results.sh regex_positive_negative`.

18.2 Codicia, grupos de captura y aserciones de contexto (*Lookaround*)

Por defecto, los cuantificadores en REGEX son **voraces** (*greedy*): consumen la mayor cantidad posible de caracteres que satisfagan la condición. En motores compatibles con PCRE o lenguajes modernos, añadir ? tras un cuantificador (como .*) lo convierte en **perezoso** (*lazy*), consumiendo el mínimo imprescindible.

Los **grupos de captura** con paréntesis (. . .) aíslan subcoincidencias para su reutilización mediante referencias hacia atrás (\1, \2).

Por último, en bioinformática es muy común buscar un motivo que deba estar precedido o seguido por una secuencia concreta, pero sin que esa secuencia sea consumida en el resultado. Para ello se emplean las construcciones de **ancho cero** (*lookaround*):

- **Positive Lookahead** (?=patrón): comprueba que lo siguiente coincida con patrón, sin incluirlo.
- **Positive Lookbehind** (?<=patrón): comprueba que lo precedente coincida con patrón, sin incluirlo.

Los cuantificadores son codiciosos por defecto en la mayoría de los motores de expresiones regulares, consumiendo la subcadena más larga posible.

Un grupo de captura (. . .) en una expresión regular aísla subcadenas para referencias hacia atrás o sustitución estructurada.

Sin escapar, el punto de una ERE coincide con cualquier carácter; \. coincide solo con el punto literal.

Se reproduce con `verification/chapter_results.sh regex_dot_escape`.

[[:alpha:]], [[:digit:]] y [[:space:]] son clases POSIX con nombre equivalentes a sus conjuntos manuales; un circunflejo al inicio de corchetes ([^0-9]) niega la clase.

Se reproduce con `verification/chapter_results.sh regex_posix_class_and_negation`.

18.3 Glob frente a regex; dialectos BRE, ERE y PCRE

Es crucial no confundir los comodines de la Shell (*globbing*) con las expresiones regulares:

- **Globbing**: lo interpreta la **Shell** para generar nombres de **archivos** antes de lanzar un comando (*.fasta).
- **Regex**: lo interpreta el **programa** (grep, sed, awk) para buscar patrones en el **texto**.

Un patrón de *glob* de Bash (*.fasta) y una expresión regular no son el mismo lenguaje: el *glob* lo interpreta la Shell sobre nombres de fichero, mientras que la expresión regular la interpreta el programa sobre el contenido de texto.

Dialecto	Dónde se usa	Características
BRE	grep básico, sed por defecto	+, ?, () y { } requieren barra: \+
ERE	grep -E, sed -E, awk	Sintaxis limpia y moderna sin escapar metacaracteres
PCRE	grep -P, utilidades avanzadas	Añade atajos (\d, \w), lookahead y no codicia

Este capítulo declara y usa siempre ERE; PCRE añade atajos de clase, cuantificadores no codiciosos y aserciones de contexto (*lookaround*); sed usa BRE por defecto y requiere -E para usar la misma sintaxis.

19 Utilidades clásicas de texto: cut, sort, uniq y tr

Unix cuenta con un repertorio de utilidades clásicas indispensables para procesar tablas y listas:

- **cut -f CAMPO**: extrae columnas de un fichero estructurado. Su delimitador por defecto ya es el tabulador, así que para un TSV basta `cut -f2`. Para otro separador se indica con `-d`, que admite **un solo carácter**: `cut -d ' ' -f2`. Escribir `cut -d '\t'` no funciona, porque ahí la barra y la `t` son dos caracteres; si necesita el tabulador explícito, la forma correcta es `cut -d $'\t' -f2`.
- **sort**: ordena líneas alfabéticamente (o numéricamente con `-n`).
- **uniq**: elimina líneas duplicadas consecutivas y cuenta frecuencias con `-c`. Compara solo líneas *adyacentes*: para eliminar duplicados que no están seguidos, ordene antes con `sort`.
- **tr**: traduce o elimina caracteres individuales. `tr 'a-z' 'A-Z'` pasa a mayúsculas; `tr -d '\r'` borra los retornos de carro que deja un fichero venido de Windows; `tr -s ' '` colapsa cada serie de espacios repetidos en uno solo. Las clases con nombre evitan depender del alfabeto: `tr '[:lower:]' '[:upper:]'` funciona igual en una máquina configurada en otro idioma.

ESTUDIO Dos utilidades más completan el repertorio mínimo. `find` recorre un árbol de directorios y selecciona por criterios en lugar de por comodín: `find datos -type f -name '*.fasta'` baja por todos los subdirectorios, cosa que `datos/*.fasta` no hace. Su compañera habitual es `xargs`, que convierte una lista de nombres en argumentos de otra orden; para que los espacios en los nombres no la rompan, la pareja se escribe `find ... -print0 | xargs -0 ....` Y `diff fichero1 fichero2` muestra en qué líneas difieren dos ficheros, que es la forma rápida de comprobar si el resultado de hoy coincide con el de la semana pasada; `diff -q` se limita a decir si son iguales o no.

En un fichero separado por tabuladores, `cut -f2` extrae el segundo campo sin necesidad de indicar delimitador; con un delimitador que no aparece en la línea, `cut` devuelve la línea completa y no emite error.

Se reproduce con `verification/chapter_results.sh cut_field`.

Comprueba. La segunda mitad de esa afirmación es la peligrosa. Si se equivoca de delimitador, `cut` no protesta: le devuelve la línea entera como si fuera el campo pedido. Prediga qué imprime `cut -d ' ' -f2` sobre una línea separada por tabuladores antes de ejecutarlo, y compruebe después que su tubería no estaba arrastrando ese error.

`uniq` solo elimina líneas duplicadas **adyacentes**; los duplicados no consecutivos requieren ordenar antes con `sort`.

Se reproduce con `verification/chapter_results.sh uniq_needs_sort`.

`tr 'a-z' 'A-Z'` traduce carácter a carácter de la primera clase a la segunda; no reconoce palabras ni límites de campo.

Se reproduce con `verification/chapter_results.sh tr_case`.

20 Transformar con sed

sed (*Stream Editor*) es un editor de flujo línea a línea. No abre una ventana interactiva; lee el texto, aplica un programa de transformación a cada línea y escribe el resultado en la salida estándar, dejando intacto el archivo original (salvo con la opción de edición en el lugar `-i .bak`).

La orden básica de sustitución es `sed 's/patrón/reemplazo/'` (cambia la primera coincidencia por línea) o `sed 's/patrón/reemplazo/g'` (con el modificador `g`, cambia todas). Si el patrón contiene barras (como en rutas de archivos), podemos usar cualquier otro delimitador: `sed 's#/usr/local#/opt#'`.

Podemos restringir las operaciones a líneas específicas mediante **direcciones**:

- **Por número o rango:** `sed '2,5s/A/T/g' fichero.txt`.
- **Última línea (\$):** `sed '$s/A/T/' fichero.txt` aplica la sustitución exclusivamente a la última línea del archivo. Aquí `$` actúa como dirección posicional de sed; dentro de una expresión regular (TAA\$) es un ancla de fin de línea, y en la Shell inicia una variable (\$VAR). Las comillas simples impiden que la Shell interprete el símbolo antes de entregarlo a sed.
- **Por coincidencia de patrón:** `sed '/^>/s/ /_/g'` (sustituye espacios solo en las cabeceras FASTA).
- **Eliminación:** `sed '/^$/d'` (elimina líneas vacías).
- **Inserción y adición:** `1i\TEXTO` (inserta antes de la línea 1) y `1a\TEXTO` (añade después).
- **Imprimir solo lo seleccionado:** `sed -n` apaga la impresión automática de cada línea, y entonces la orden `p` imprime únicamente las que usted elija. `sed -n '5p'` muestra la quinta línea; `sed -n '/^>/p'` muestra solo las cabeceras. Sin `-n`, esas mismas órdenes imprimirían las líneas seleccionadas *dos veces*, que es el desconcierto habitual de la primera vez.
- **Negar una selección:** el signo de admiración invierte la dirección. `sed -n '/^>/!p'` imprime todo *menos* las cabeceras, es decir, solo la secuencia; y `sed '1!d'` conserva la primera línea y borra el resto.

sed es un editor de flujo: lee su entrada línea a línea, aplica el programa de edición y escribe el resultado en stdout sin modificar el fichero de entrada a menos que se use `-i`.

`sed 's/patrón/reemplazo/'` sustituye únicamente la primera coincidencia de cada línea; las siguientes permanecen intactas.

Se reproduce con `verification/chapter_results.sh sed_first_match_only`.

`sed 's/patrón/reemplazo/g'` sustituye todas las coincidencias de cada línea gracias al indicador global `g`.

Se reproduce con `verification/chapter_results.sh sed_global_flag`.

sed acepta un delimitador distinto de `/` para la sustitución, evitando escapar las barras de una ruta.

Se reproduce con `verification/chapter_results.sh sed_custom_delimiter`.

`sed '2s/.../.../'` actúa solo en la línea 2; con un rango de líneas actúa en ese rango; con una dirección por patrón actúa en las líneas que coinciden; sin dirección se aplica a todas.

Se reproduce con `verification/chapter_results.sh sed_addressing`.

`sed '/^$/d'` elimina las líneas completamente vacías de la salida; las demás pasan sin cambios.

Se reproduce con `verification/chapter_results.sh sed_delete_blank_lines`.

`sed -i` sobrescribe el fichero de entrada sin guardar copia a menos que se especifique un sufijo de seguridad (`sed -i .bak`); usarlo sin sufijo hace irreversible cualquier error en el patrón.

Se reproduce con `verification/chapter_results.sh sed_inplace_backup`.

Las órdenes estructurales de sed para borrar (`d`), insertar (`i`), añadir (`a`) y cambiar líneas completas (`c`) permiten transformar el flujo de texto de forma programática.

Se reproduce con `verification/chapter_results.sh sed_delete_insert_append_change`.

20.1 La tríada Unix: roles complementarios

`grep` decide qué líneas pasan sin cambiarlas (filtra); `sed` transforma el contenido de cada línea que pasa (transforma); `awk` divide cada línea en campos y aplica lógica de programa (procesa). Encadenar los tres cubre la mayoría de las tareas de procesamiento de texto en bioinformática.

21 Procesar texto estructurado con `awk`

`awk` (creado por Aho, Weinberger y Kernighan) es un lenguaje de programación completo orientado al procesamiento de datos estructurados en filas (**registros**) y columnas (**campos**). Su estructura básica sigue el paradigma: patrón { acción }.

Variables especiales fundamentales:

- `$0`: contenido completo de la línea actual.
- `$1, $2, ..., $NF`: primer, segundo y último campo de la línea actual.
- `NF`: número total de campos en la línea actual (*Number of Fields*).
- `NR`: número de registro o línea global actual (*Number of Records*).
- `FS`: separador de campos (se define con `-F ' , '` o `-F '\t'`).
- `FNR`: número de registro *dentro del fichero actual*. Con un solo fichero coincide con `NR`; con varios, `NR` sigue contando y `FNR` vuelve a empezar. De ahí el modismo `FNR == 1` para hacer algo en la primera línea de cada fichero, por ejemplo saltarse una cabecera repetida.
- `FILENAME`: nombre del fichero que se está leyendo, imprescindible cuando la salida mezcla varios y hay que saber de dónde vino cada fila.
- `OFS` y `ORS`: separador de campos y de registros *de salida*. `FS` dice cómo se lee; `OFS` cómo se escribe. Para convertir un CSV en un TSV basta el programa de una línea que aparece bajo esta lista: la asignación aparentemente inútil del primer campo sobre sí mismo obliga a `awk` a reconstruir la línea con el nuevo separador.
- `RS`: separador de registros de entrada. Fijándolo al símbolo mayor que, un fichero FASTA se lee por secuencias en lugar de por líneas, que es la forma natural de tratar un formato multilínea.
- **Bloques BEGIN y END**: `BEGIN { ... }` se ejecuta una única vez antes de leer los datos; `END { ... }` se ejecuta una única vez tras procesar la última línea.

Y tres construcciones del lenguaje que hacen falta en cuanto el programa pasa de una línea. La condicional se escribe como en cualquier lenguaje, `if ($3 > 100) { ... } else { ... }`, y puede ir dentro de la acción. `split(cadena, destino, separador)` parte una cadena en un array y devuelve cuántos trozos salieron, que es como se desmenuza el noveno campo de un GFF. Y `index(cadena, subcadena)` devuelve la posición de la primera aparición, o cero si no está, sin necesidad de expresión regular.

```
awk 'BEGIN { FS = ","; OFS = "\t" } { $1 = $1; print }' muestras.csv > muestras.tsv
```

Concepto. Cuando el programa `awk` pasa de tres o cuatro líneas, sáquelo del entrecomillado y guárdelo en un fichero con extensión `.awk`, que se ejecuta con `awk -f programa.awk datos.tsv`. Gana comentarios, sangrado y control de versiones, y deja de pelearse con las comillas de la Shell. Es el mismo salto que dio el capítulo al pasar de la orden suelta al script.

En `awk`, `NR` es el número de registro actual, `NF` es el número de campos y `$0` es la línea completa; un bloque `END` se ejecuta exactamente una vez tras procesar toda la entrada.

Se reproduce con `verification/chapter_results.sh awk_nr_nf`.

`awk -F ' , '` cambia el separador de campos de espacios/tabuladores a coma para procesar ficheros CSV.

Se reproduce con `verification/chapter_results.sh awk_custom_field_separator`.

`awk '{suma += $1} END {print suma}'` acumula valores aritméticos de una columna a lo largo de las líneas y emite el total final en el bloque `END`.

Se reproduce con `verification/chapter_results.sh awk_arithmetic_accumulate`.

El operador `~` comprueba si un campo coincide con una expresión regular (`$1 ~ /^A/`); `!~` comprueba que no coincida.

Se reproduce con `verification/chapter_results.sh awk_match_operator`.

`length($0)` devuelve la longitud de la línea; `substr($0, pos, len)` extrae una subcadena a partir de la posición `pos`.

Se reproduce con `verification/chapter_results.sh awk_length_substr`.

`sub(patrón, reemplazo, $0)` sustituye la primera coincidencia en el registro; `gsub(patrón, reemplazo, $0)` sustituye todas.

Se reproduce con `verification/chapter_results.sh awk_sub_gsub`.

`awk -v nombre=valor '...'` declara una variable accesible dentro del programa `awk` con el valor asignado desde la Shell.

Se reproduce con `verification/chapter_results.sh awk_dash_v`.

Un array asociativo de `awk` (`count[$clave]++`) acumula frecuencias por clave sin necesidad de declarar previamente qué claves existirán.

Se reproduce con `verification/chapter_results.sh awk_assoc_array`.

21.1 Ejemplo trabajado: medir exones en un GFF

ESTUDIO El formato GFF anota regiones sobre un genoma. Cada línea tiene nueve campos separados por tabuladores: secuencia, origen, tipo, inicio, fin, puntuación, hebra, fase y un noveno campo con atributos en la forma `clave=valor` separados por punto y coma. Las líneas que empiezan por almohadilla son cabeceras. El fichero `data/tc02_ anotacion.gff` contiene una anotación reducida con tres genes y sus exones.

La pregunta. Cuántos exones hay, cuánto suman y cuál es su longitud media.

La hipótesis. Las coordenadas de un GFF son cerradas y empiezan en 1, así que la longitud de una región es $fin - inicio + 1$, no $fin - inicio$. Con seis exones de unos trescientos o cuatrocientos pares de bases, la suma debería rondar los dos millares.

```
awk -F'\t' '$3 == "exon" { n++; suma += $5 - $4 + 1 }
END { printf "EXONES:%d LONGITUD_TOTAL:%d MEDIA:%.1f\n", n, suma, suma/n }' \
data/tc02_ anotacion.gff
```

```
EXONES:6 LONGITUD_TOTAL:2186 MEDIA:364.3
```

La comprobación. Seis exones es lo que se cuenta a mano en el fichero, y la media cae dentro de lo previsto. Si hubiéramos olvidado el `+ 1` la suma habría sido 2180: un error de seis bases que ningún mensaje habría señalado y que en un genoma completo se convierte en miles.

El segundo paso. Ahora el nombre de cada gen, que vive dentro del noveno campo y hay que desmenuzar. Se parte primero por punto y coma para aislar el atributo `Name`, y después por el signo igual para quedarse con su valor:

```
awk -F'\t' '$3 == "gene" { split($9, campos, ";"); split(campos[2], nombre, "=")
printf "%s\t%d\n", nombre[2], $5 - $4 + 1 }' \
data/tc02_ anotacion.gff
```

```
BRCA1x      2201
TP53x       901
MIRx        451
```

Atención. Este segundo programa da por hecho que Name es el segundo atributo del noveno campo. Es cierto en este fichero y falso en cuanto cambie de proveedor de anotación. Lo correcto en producción es recorrer los atributos y quedarse con el que se llama Name, no con el que ocupa la segunda posición. Que un programa funcione sobre su fichero de prueba no significa que funcione sobre el fichero siguiente, y esa distinción es la diferencia entre un análisis y un accidente.

Sobre la anotación reducida del capítulo, los seis exones suman 2186 bases con una media de 364,3, y las longitudes de los tres genes son 2201, 901 y 451 bases.

Se reproduce con `verification/chapter_results.sh awk_gff_exones`.

21.2 El puente de Shell a awk: conteo de nucleótidos

Para la secuencia AACG, el programa de conteo por bases imprime A:2 C:1 G:1 T:0, incluidas las bases ausentes con valor cero.

```
echo AACG | awk '{ for (i = 1; i <= length($0); i++) c[substr($0, i, 1)]++ }
                END { for (i = 1; i <= 4; i++) { s = substr("ACGT", i, 1)
                printf "%s:%d ", s, c[s] + 0 }
                print "" }'
```

Observe el + 0 del printf. Sin él, una base que no aparece en la secuencia no tiene entrada en el array y se imprimiría como cadena vacía en lugar de como cero. Es la diferencia entre una tabla que dice «no hay timinas» y otra que deja un hueco donde el lector no sabe si falta el dato o falta la base.

Se reproduce con `verification/chapter_results.sh bridge_counts`.

21.3 Instalar programas: dónde se trata

REFERENCIA Antes o después necesitará instalar una herramienta que no viene con el sistema, como BLAST. No se trata aquí porque el procedimiento depende de la distribución y del permiso que tenga en la máquina, y porque conviene verlo cuando la herramienta hace falta de verdad. El capítulo dedicado a la búsqueda por similitud lo aborda con BLAST como caso, incluidas las instalaciones sin privilegios de administrador y los entornos aislados. Si necesita adelantarlo, la guía de laboratorio de la asignatura tiene el procedimiento para los equipos del aula.

22 Lo que viene después: de procesar texto a razonar sobre su coste

A lo largo de este capítulo hemos dominado la construcción de tuberías, el uso de expresiones regulares y la automatización mediante scripts en Bash y awk. Hemos resuelto el problema de procesar miles de registros biológicos de manera reproducible y sin intervención manual.

Sin embargo, a medida que los conjuntos de datos en biomedicina crecen desde unos pocos megabytes hasta cientos de gigabytes, surge una pregunta crítica: ¿cuánto tiempo y memoria consume cada una de estas soluciones? ¿Por qué ordenar con `sort` antes de `uniq` es rápido para diez mil líneas pero lento para cien millones? Responder a estas preguntas requiere pasar de la mera automatización al **pensamiento algorítmico y al análisis de complejidad**, que es el núcleo fundamental que exploraremos en el capítulo TC-CH03 (Algoritmos y complejidad).

Este capítulo y TC-CH01 preparan explícitamente la transición a TC-CH03: una vez que se sabe automatizar y procesar texto, la pregunta abierta es cuánto cuesta cada construcción cuando el tamaño de los datos crece, que es el objeto de estudio de TC-CH03.

23 Método y errores frecuentes

Este capítulo recupera, sin volver a enseñarlo, el ciclo predecir/probar en pequeño/inspeccionar/contrastar/registrar introducido en TC-CH01; aquí se aplica a tuberías, patrones y scripts en vez de a órdenes sueltas.

Síntoma	Causa probable	Comprobación y corrección
grep -c da menos de lo esperado	cuenta líneas, no coincidencias	comparar con grep -o ... wc -l
uniq deja duplicados	no eran adyacentes	sort antes de uniq
[AG]+ no coincide	se usó BRE en vez de ERE	usar grep -E y declarar el dialecto
un bucle for no encuentra ficheros	no había coincidencias y falta nullglob	activar shopt -s nullglob y comprobar longitud del array
una tubería parece correcta pese a un fallo	solo se observó el estado del último eslabón	set -o pipefail y probar cada etapa por separado
sed no cambia todas las coincidencias	falta el indicador g	añadir g al final de s///
una función no «devuelve» el texto esperado	se confundió return (estado numérico) con capturar salida	usar \$(función) para texto, return solo para estado

Los síntomas de esta tabla corresponden a los errores de composición más frecuentes observados al automatizar procesamiento de texto con las herramientas de este capítulo.

24 Síntesis de referencia

Necesidad	Construcción principal	Pregunta de control
parametrizar	NOMBRE=valor, \$NOMBRE, \$(orden)	¿hay espacios alrededor del =?
citar con seguridad	comillas simples/dobles, \	¿quiero expansión o texto literal?
componer	, >, >>, 2>	¿qué recibe y produce cada etapa?
decidir	[[]], if/elif/else	¿comparo como texto o como número?
repetir	for, while, break/continue	¿sé de antemano cuántas veces?
reutilizar	funciones, return frente a \$(función)	¿necesito el estado o el texto?
buscar	grep -n -i -v -c -o -E -A -w -e -r -F	¿cuento líneas o coincidencias y con qué dialecto?
describir patrones	anclas, clases, cuantificadores, grupos	¿el caso negativo también falla como espero?
transformar	sed s///[g], direcciones, d/i/a/c, -i	¿sustituyo la primera coincidencia o todas?
procesar por registros	awk: NR/NF/FS, funciones, ~	¿qué separa un registro de un campo aquí?

La tabla de síntesis enlaza cada necesidad operativa del capítulo con su construcción principal y una pregunta de control verificable.

25 Glosario esencial

- **Metacaracter:** carácter con significado especial para la Shell antes de ejecutar la orden.
- **stdin/stdout/stderr:** entrada, salida y error estándar.
- **Tubería:** conexión del stdout de un proceso al stdin de otro.
- **Función:** bloque de órdenes con nombre, reutilizable, con argumentos propios.
- **Bucle:** estructura que repite órdenes mientras se cumple una condición o para una lista de elementos.
- **Expansión de llaves:** generación de texto literal a partir de { . . . , . . . } o {n . . m}, sin consultar el sistema de ficheros.

- **Glob:** patrón de Bash para nombres de fichero, interpretado por la Shell.
- **Regex (ERE):** lenguaje de patrones sobre el contenido de un texto, sintaxis extendida usada con `grep -E`.
- **Grupo de captura:** subpatrón entre paréntesis cuyo texto coincidente puede referenciarse después.
- **Cuantificador voraz (*greedy*):** cuantificador que coincide con la cadena más larga posible.
- **Aserción de ancho cero (*lookaround*):** comprobación de contexto que no consume caracteres en el resultado.
- **Stream editor (*sed*):** editor que procesa texto línea a línea sin modificar el origen salvo petición explícita.
- **Registro / Campo (*awk*):** fila y columna en el modelo de datos estructurados de `awk`.

26 Fuentes y límites de uso

Este capítulo se apoya en las presentaciones docentes de la asignatura sobre programación de la Shell y expresiones regulares, escritas por Álvaro Serrano Navarro. El comportamiento de las opciones de seguridad, las expansiones y los operadores de prueba está tomado del manual de referencia de Bash [4]; el de las utilidades de texto, de los manuales de GNU [6, 3, 7, 5]. La descripción del formato FASTA sigue la especificación del NCBI [8]. La organización de proyecto y el criterio de reproducibilidad siguen a Buffalo [2], y los patrones de scripting científico, a Allesina y Wilmes [1].

Los datos de `data/` son sintéticos y de elaboración propia, con licencia CC0 y manifiesto de sumas propio; están descritos en `data/README.md`. Todas las órdenes impresas se reproducen con `verification/` tal como están escritas.

Bibliografía

- [1] Stefano Allesina and Madlen Wilmes. *Computing Skills for Biologists: A Toolbox*. Princeton University Press, 2019. Ejemplos de línea de comandos contrastados en el PDF local; no sostienen ninguna afirmación en solitario.
- [2] Vince Buffalo. *Bioinformatics Data Skills: Reproducible and Robust Research with Open Source Tools*. O'Reilly Media, 2015. Pasajes comprobados en las páginas físicas 74–79 y 170–180 del PDF local.
- [3] GNU Project. *GNU Grep 3.12 Manual*, 2025. Sintaxis BRE/ERE y opciones de `grep`. Consultado el 7 de agosto de 2026. <https://www.gnu.org/software/grep/manual/grep.html>.
- [4] GNU Project. *Bash Reference Manual*, 2026. Consultado el 7 de agosto de 2026. <https://www.gnu.org/software/bash/manual/bash.pdf>.
- [5] GNU Project. *GNU Awk User's Guide*, 2026. Consultado el 7 de agosto de 2026. https://www.gnu.org/software/gawk/manual/html_node/index.html.
- [6] GNU Project. *GNU Coreutils Manual*, 2026. Documentación de `cut`, `sort`, `uniq` y `tr`. Consultada el 7 de agosto de 2026. <https://www.gnu.org/software/coreutils/manual/coreutils.html>.
- [7] GNU Project. *GNU sed User's Guide*, 2026. Sintaxis de `s///`, direcciones y `-i`. Consultado el 7 de agosto de 2026. <https://www.gnu.org/software/sed/manual/sed.html>.
- [8] National Center for Biotechnology Information. *FASTA Format for Nucleotide Sequences*, 2026. Documentación oficial. Consultada el 7 de agosto de 2026. <https://www.ncbi.nlm.nih.gov/genbank/fastaformat>.